Multi-Dimensional Apriori-Window: Exact Mining of Dense Itemset Regions over Arbitrary Dimensions

1st Given Name Surname
Dept. Name of Organization
Name of Organization
City, Country
email@example.com

2nd Given Name Surname
Dept. Name of Organization
Name of Organization
City, Country
email@example.com

***Abstract*—In large-scale retail, epidemiology, and logistics data, analysts need to answer both *what* is dense (itemsets) and *when and where* it is dense (spatio-temporal regions) simultaneously. However, existing methods cannot achieve both: pattern mining methods do not output regions, while region extraction methods (e.g., M-Zoom) cannot be integrated with itemset enumeration due to Cartesian-product block constraints and lack of anti-monotonicity. This paper proposes Multi-Dimensional Apriori-Window, which exactly extracts all dense itemsets together with their dense regions over arbitrary d-dimensional grids. By reducing the multi-dimensional problem to a duplicate-aware one-dimensional problem for each fixed spatial start point, and by exploiting the anti-monotonicity of windowed support counts for Apriori pruning, the method exactly enumerates all dense patterns together with their arbitrarily-shaped dense regions.**

The proposed method exactly extracts all dense patterns and their dense regions by construction (F1 $= 1.00$, Jaccard $= 1.00$). Comparative experiments on synthetic data with diverse embedded dense-region shapes show that pattern mining baselines achieve pattern detection F1 of only 0.356–0.917, with substantial variation depending on the region shape. Region extraction baselines (MZoom, LISA, ST-DBSCAN), even when given GT patterns, achieve mean Jaccard $\leq$ 0.190, failing to closely approximate the sliding-window dense regions. Scalability experiments confirm linear scaling over five orders of magnitude ($N = 10^3$–10^8 transactions), processing 10^8 transactions within 10 minutes. Application to d=3–4 real-world data from crime, complaints, and earthquake domains yields interpretable insights consistent with domain knowledge.

***Index Terms*—Frequent pattern mining, multi-dimensional data, sliding window, dense region detection, Apriori, interval representation**

I. Introduction

A. Motivation and Application Background

A data analyst at a large retail chain wants to know "which product combinations are intensively purchased, at which store locations, and during which time periods." For example, from 100 stores $\times$ 52 weeks of point-of-sale data, the insight that "beer and potato chips are concentrated during weeks 46–52 (year-end season) at urban stores (stores 0–9)" directly informs region-specific inventory planning and promotional strategies. Similarly, in epidemiology, co-occurrence patterns of symptoms concentrated in specific regions and periods become intervention targets; in logistics, regional and temporal surges in shipment volumes support service optimization.

Multi-dimensional data are thus accumulated across many domains, and extracting *spatio-temporally localized dense patterns* from such data can contribute to knowledge discovery beyond what conventional aggregate analysis provides.

B. Limitations of Existing Methods

In the above applications, what analysts truly need is the pair (i) *what* is dense (itemset P) and (ii) *when and where* it is dense (d-dimensional dense region), i.e., the tuple $(P, R(P))$. Pattern P alone leaves "which store to reorder to and when" unknown, while region R alone leaves "what to reorder" unspecified. Only when both are available can region- and period-specific strategies be formulated. However, no existing method provides both simultaneously.

a) Pattern mining methods.: Frequent pattern mining [1], [2] relies on global support and thus misses locally dense patterns. Existing methods that consider locality [3] remain limited to a one-dimensional design on the temporal axis and cannot exploit multi-dimensional spatial information.

b) Region extraction methods.: Dense subtensor mining (M-Zoom [4], etc.) cannot fit arbitrarily-shaped dense regions due to Cartesian-product block constraints, and its pattern enumeration is inaccurate (detailed in §II). Spatial statistics methods [5], [6] cannot handle the temporal dimension. Scan statistics [7] lack the concept of itemsets. ST-DBSCAN [8] relies on point-neighborhood density and thus cannot detect localized concentrations distinguished from background density; it also lacks anti-monotonicity and therefore cannot be integrated with Apriori pruning.

C. Contributions of This Work

This paper proposes **Multi-Dimensional Apriori-Window**, which exactly extracts all dense patterns and their dense regions over arbitrary d-dimensional grids (time + $d-1$ spatial/categorical dimensions) (§IV). In a nutshell, the proposed method enables analysts to **exactly extract "when, where,**

and what sells" under the sliding-window density definition.

The technical essence of this work lies in extending the dense interval computation of one-dimensional Apriori-Window [3] to d dimensions by applying it to the temporal multiset for each fixed spatial start point. That is, even after multi-dimensionalization, the core computation is essentially a *duplicate-aware one-dimensional Apriori-Window*.

The following three techniques realize this multi-dimensionalization.

1) **Reduction to 1D per fixed spatial start point**: For each spatial start point $\boldsymbol{s}_{\mathrm{sp}}$, occurrences of a pattern falling into the spatial window are collapsed into a temporal multiset $L_P(\boldsymbol{s}_{\mathrm{sp}})$. This decomposes the d-dimensional problem into multiple one-dimensional problems keyed by spatial start points.
2) **Apriori pruning via singleton interval map intersection**: Dense regions of each item are maintained as temporal interval sequences per spatial start point. For candidate patterns, these are intersected to form candidate intervals. This enables interval-based pruning isomorphic to the 1D case for $k \geq 2$ (Lemma 1).
3) **Duplicate-aware t-axis stride**: In two or more dimensions, duplicate timestamps arise. Instead of maintaining dense-start positions one by one, contiguous blocks of left endpoints guaranteed to be dense by the current window alone are processed in bulk, and output as dense intervals (Lemma 2).

II. Related Work

This work addresses the problem of discovering dense itemset regions from spatio-temporal transaction data. This problem lies at the intersection of two research lineages: (1) the *pattern mining lineage* aimed at enumerating itemsets, and (2) the *dense block detection lineage* aimed at detecting dense blocks in multi-dimensional spaces. Below we organize both lineages and clarify why this work is positioned at their intersection.

A. Pattern Mining Lineage

a) Frequent Itemset Mining (FIM).: The Apriori algorithm by Agrawal et al. [1] exploits the monotonicity of itemsets (subsets always have support no less than their supersets) to drastically prune the candidate space. FP-Growth by Han et al. [2] further accelerates mining using prefix trees. These methods use global support over the entire static transaction database as their criterion, and thus suffer from the fundamental problem of missing locally dense patterns [3].

b) Local dense interval detection methods.: Several methods based on *intervals* between consecutive occurrence timestamps have been proposed for detecting locally dense intervals. LPFIM [9] splits intervals when the gap between two consecutive occurrences exceeds a threshold minthd1, and outputs intervals whose local support rate exceeds a threshold and whose length exceeds minthd2. LPPM [10] introduces spillover accumulation for noise-robust interval detection.

These methods estimate density by evaluating "the time difference between two consecutive occurrences (inter-occurrence interval)." However, as Takahashi et al. [3] pointed out, since the gap parameter governs both pattern identification accuracy and interval detection accuracy, parameter settings that simultaneously keep both high generally do not exist—a structural limitation. Moreover, these methods are *inherently non-extensible to multiple dimensions*. Occurrence points on a d-dimensional grid have no linear ordering, making "the interval between two consecutive occurrences" ill-defined.

Apriori-Window [3] is an exact method that resolves this structural limitation. By *directly* computing the number of occurrences within a sliding window $[l, l+W]$ rather than via inter-occurrence intervals, it evaluates local frequency without requiring a gap parameter. Combined with anti-monotonicity-based Apriori pruning and t-axis stride, it exhaustively and exactly enumerates dense intervals. Comparative experiments demonstrated performance significantly exceeding LPFIM and LPPM [3].

Why multi-dimensionalization is possible: The definition "number of occurrences within a rectangular window" naturally extends to d dimensions (simply replacing the window with a d-dimensional hyperrectangle $\prod_k [s_k, s_k+W_k]$). In contrast to interval-based methods that depend on linear ordering, the design of Apriori-Window does not require linear ordering. This work builds on this observation to extend Apriori-Window from one dimension to arbitrary d dimensions.

c) Colocation mining.: Colocation mining [11] evaluates patterns where multiple feature types co-exist in geographic proximity using the Participation Index (PI). Qian et al. [12] extended this to weighted sliding windows (STCP-Miner). These methods deal with spatial proximity-based co-occurrence rather than transaction-based co-occurrence, and do not target dense region extraction. However, comparison in terms of pattern identification accuracy (F1) is possible (§V-A).

B. Region Extraction Lineage

a) Dense block detection methods.: Dense block detection methods including M-Zoom [4] and others (tensor: [13]–[17]; graph: SpecGreedy [18]) detect dense blocks in multi-dimensional data. They select value sets $B_k \subseteq D_k$ along each dimension and perform greedy search to maximize the mass-to-volume ratio of the Cartesian-product block $\prod_k B_k$.

These methods have two structural limitations.

First, output is restricted to Cartesian-product blocks. For irregularly-shaped dense regions, they inevitably include excessive areas or miss dense cells, resulting in low Jaccard even when GT patterns are provided (demonstrated in §V-A).

Second, pattern enumeration is inaccurate. While adding items as an additional dimension allows enumeration to be attempted, the Cartesian-product constraint causes all subsets of the itemset C_B within a block to be assumed dense in the same region, generating false positives (demonstrated in §V-A).

TABLE I: Comparison with related methods. "Itemset" refers to pattern enumeration capability; "Dense region" refers to the ability to output spatio-temporally localized dense regions.

Method	Item-set	ST dense region	Region shape
Apriori [1]	✓	×	—
LPFIM [9] et al.	✓	1D only	Interval
Apriori-Window [3]	✓	1D only	Interval
Colocation [11]	✓	×	—
STCP-Miner [12]	✓	×	—
M-Zoom [4] et al.	△*	✓	Cartesian only
SaTScan [7]	×	✓	Circle/cylinder
FlexScan [20]	×	✓	Irregular
LISA [5] / G_i^* [6]	×	Spatial only	Arbitrary
ST-DBSCAN [8] et al.	×	✓	Arbitrary
Ours	✓	✓	Arbitrary

*Patterns can be output by including items as a dimension, but Cartesian constraints cause false positives and preclude region separation.

b) Spatial statistics.: LISA [5] identifies spatial hotspots using local Moran's I, and Getis-Ord G_i^* [6] evaluates local concentration via z-scores. Both target *spatial dimensions only*; running them per time slice loses temporal continuity.

c) Spatial and spatio-temporal scan statistics.: Kulldorff's [7], [19] spatial/spatio-temporal scan statistics detect significant clusters using circular/cylindrical scanning windows. FlexScan [20] extends this to irregularly-shaped windows.

All of the above statistical methods are designed to detect per-variable anomalous concentrations and lack the ability to enumerate co-occurrence patterns (itemsets) within transactions. Enumerating all patterns would require $O(2^{|\mathcal{I}|})$ independent runs, and integration with Apriori pruning is not part of their design.

d) Spatio-temporal density-based clustering (ST-DBSCAN, etc.).: ST-DBSCAN [8] extends DBSCAN [21] to spatio-temporal data with independent thresholds $\varepsilon_1, \varepsilon_2$ for spatial and temporal distances. While it outputs arbitrarily-shaped clusters, first, it defines density by point-neighborhood counts; when occurrences are spatially uniform, the entire dataset becomes a single cluster, failing to identify localized concentrations. Second, point-neighborhood density lacks anti-monotonicity, making independent processing of all patterns $O(2^{|\mathcal{I}|})$ unavoidable.

All region extraction methods above lack the concept of itemsets, but can be evaluated as region extractors when GT patterns and their occurrence densities are provided (§V-A).

C. Positioning of This Work

The above analysis shows that no existing method simultaneously achieves pattern enumeration and multi-dimensional dense region extraction. This work exploits the anti-monotonicity of windowed-occurrence-based density to naturally integrate Apriori pruning, realizing both within a single framework. Table I illustrates this positioning.

III. Problem Definition

This section reviews the one-dimensional setting of Apriori-Window [3] (§III-A) and then formulates its d-dimensional extension (§III-B).

A. Apriori-Window (One-Dimensional)

Let $\mathcal{I} = \{i_1, \ldots, i_m\}$ be the item universe and $\mathcal{D}_{1\mathrm{D}} = \{(t_n, d_n)\}_{n=1}^{N}$ be a timestamped transaction sequence, where $t_n \in \mathbb{Z}_{\geq 0}$ is an integer timestamp and $d_n \subseteq \mathcal{I}$ is an itemset.

The occurrence sequence of a pattern $P \subseteq \mathcal{I}$ is

$$\mathrm{occ}(P) = \{t_n : P \subseteq d_n\}.$$

Definition 1 (Interval support [3])**.** *Under window width W ($W > 0$), the interval support of pattern P at window left endpoint l is defined as*

$$\sup(P,\, l,\, W) = \big|\{t \in \mathrm{occ}(P) : l \leq t \leq l + W\}\big|.$$

The window $[l, l+W]$ contains $W+1$ discrete time points.

Definition 2 (Dense-start set and dense interval)**.** *For a support threshold θ,*

$$\mathcal{T}_\theta(P) = \{l \in \mathbb{Z}_{\geq 0} : \sup(P, l, W) \geq \theta\}$$

is the dense-start set *of P.*

Let $[a, b] \subseteq \mathcal{T}_\theta(P)$ be a maximal contiguous component. Then $[a,\, b+W]$ is called a dense interval *of P.*

B. Extension to d Dimensions

Let $\mathcal{D} = \{(\boldsymbol{b}_n, d_n)\}_{n=1}^{N}$ be a d-dimensional transaction sequence, where

$$\boldsymbol{b}_n = (b_n^{(0)}, b_n^{(1)}, \ldots, b_n^{(d-1)}) \in \prod_{k=0}^{d-1} \{0, 1, \ldots, N_k - 1\}$$

is an integer coordinate vector (axis 0 is time) and $d_n \subseteq \mathcal{I}$ is an itemset. N denotes the *number of transactions*.

Definition 3 (Temporal multiset reduction to a fixed spatial start point)**.** *Fix a spatial start vector*

$$\boldsymbol{s}_{\mathrm{sp}} = (s_1, \ldots, s_{d-1})$$

and spatial window widths $\boldsymbol{W}_{\mathrm{sp}} = (W_1, \ldots, W_{d-1})$ (each $W_k > 0$). The spatial window along axis k, $[s_k, s_k + W_k]$, contains $W_k + 1$ discrete coordinates. The temporal multiset of pattern P is defined as

$$L_P(\boldsymbol{s}_{\mathrm{sp}}) = \{\{b_n^{(0)} \mid P \subseteq d_n,\ s_k \leq b_n^{(k)} \leq s_k + W_k\ (\forall k = 1, \ldots, d-1)\}\}$$

where $\{\{\cdot\}\}$ denotes a multiset allowing duplicates.

The essence of this definition is that, for each fixed spatial start point, the d-dimensional problem is reduced to a *one-dimensional problem with duplicate timestamps*.

Definition 4 (d-dimensional sliding window support)**.** *For a temporal start point s_0 and spatial start vector $\boldsymbol{s}_{\mathrm{sp}}$, the support of pattern P is defined as*

$$S_P(s_0, \boldsymbol{s}_{\mathrm{sp}}) = \big|\{t \in L_P(\boldsymbol{s}_{\mathrm{sp}}) : s_0 \leq t \leq s_0 + W_0\}\big|. \quad (1)$$

Definition 5 (Dense-start set and dense region)**.** *For a fixed spatial start vector* $\boldsymbol{s}_{\mathrm{sp}}$*,*

$$\mathcal{T}_\theta(P, \boldsymbol{s}_{\mathrm{sp}}) = \{s_0 \in \mathbb{Z}_{\geq 0} : S_P(s_0, \boldsymbol{s}_{\mathrm{sp}}) \geq \theta\}$$

is the dense-start set *of* P *at* $\boldsymbol{s}_{\mathrm{sp}}$*.*

Let $[a_j, b_j]$ *be a maximal contiguous component of* $\mathcal{T}_\theta(P, \boldsymbol{s}_{\mathrm{sp}})$*. Then* $[a_j,\ b_j + W_0]$ *is called a* dense interval *of* P *at* $\boldsymbol{s}_{\mathrm{sp}}$ *(a natural extension of Definition 2).*

The collection of dense intervals over all spatial start points,

$$\mathcal{R}_\theta(P) = \{(\boldsymbol{s}_{\mathrm{sp}},\ [a_j,\ b_j + W_0])\},$$

is called the dense region *of* P*.*

a) Apriori monotonicity.:

Lemma 1 (Monotonicity of dense-start sets)**.** *For any pattern* P *and its subset* $Q \subset P$*, and any spatial start vector* $\boldsymbol{s}_{\mathrm{sp}}$*,*

$$\mathcal{T}_\theta(P, \boldsymbol{s}_{\mathrm{sp}}) \subseteq \mathcal{T}_\theta(Q, \boldsymbol{s}_{\mathrm{sp}})$$

holds.

The proof is given in Appendix A.

By Lemma 1, the dense-start set of pattern P is contained within the dense intervals of each singleton item $i \in P$. The proposed method exploits this property by using *intersection of singleton interval maps* as candidate intervals (§IV-C).

C. d-Dimensional Dense Region Detection Problem

The d-dimensional dense region detection problem is, given a d-dimensional transaction sequence $\mathcal{D}$, window widths $\boldsymbol{W} = (W_0, W_1, \ldots, W_{d-1}) \in \mathbb{Z}^d_{>0}$, and a support threshold θ, to find the complete set of dense regions $\mathcal{R}_\theta(P) = \{(\boldsymbol{s}_{\mathrm{sp}}, [a_j,\ b_j + W_0])\}$ for each pattern P. The window along each axis k is a closed interval $[s_k, s_k + W_k]$ ($W_k + 1$ bins).

IV. Proposed Method: Multi-Dimensional Apriori-Window Algorithm

We propose the **Multi-Dimensional Apriori-Window algorithm** to solve the problem in §III-C.

A. 1D Multiset Reduction via Fixed Spatial Start Points

For a pattern P and a fixed spatial start point $\boldsymbol{s}_{\mathrm{sp}}$, we construct the temporal multiset $L_P(\boldsymbol{s}_{\mathrm{sp}})$ from Definition 3. This is obtained by distributing each occurrence transaction of P to all spatial start points whose window contains that transaction.

a) Distribution to spatial start points.: When a transaction containing pattern P resides at discrete coordinates

$$\boldsymbol{b} = (b^{(0)}, b^{(1)}, \ldots, b^{(d-1)}),$$

the k-th component ($k \geq 1$) of a spatial start point containing this transaction satisfies

$$s_k \in [\max(0,\ b^{(k)} - W_k),\ b^{(k)}].$$

Therefore, one occurrence contributes to at most

$$W_{\mathrm{sp}} = \prod_{k=1}^{d-1} (W_k + 1)$$

spatial start points.

b) Construction cost.: Let N_P denote the number of occurrence transactions of pattern P. Construction costs $O(N_P \cdot W_{\mathrm{sp}})$; including sorting, the total is $O(\widetilde{N}_P \log \widetilde{N}_P)$ where $\widetilde{N}_P = N_P \cdot W_{\mathrm{sp}}$.

B. Duplicate-Aware Interval Solver on the t-Axis

For a fixed spatial start point $\boldsymbol{s}_{\mathrm{sp}}$, we compute dense intervals from the sorted multiset $L = L_P(\boldsymbol{s}_{\mathrm{sp}})$. In two or more dimensions, multiple transactions may share the same timestamp, producing duplicates in L. Consequently, the simple "+1 jump" of the one-dimensional case cannot be directly applied.

The key idea is a two-branch stride governed by the comparison of the window count c and the threshold θ (Figure 1). When $c < \theta$ (non-dense), the left endpoint l jumps to the position where the next event enters the window, skipping unnecessary scans (Eq. (2), Figure 1(b)). When $c \geq \theta$ (dense), the surplus $c - \theta$ determines a contiguous block $[l, l_{\mathrm{dense}}]$ of left endpoints that are all guaranteed to be dense, and this entire block is output as a dense interval in bulk (Eq. (3), Figure 1(a)). The correctness of this bulk processing is guaranteed by Lemma 2.

Specifically, the left endpoint update in the non-dense case is

$$l \leftarrow \max(l + 1,\ L[\mathrm{next}] - W_0) \tag{2}$$

where $L[\mathrm{next}]$ is the first event outside the right boundary of the current window. In the dense case, the block right boundary is

$$l_{\mathrm{dense}} = L[l_{\mathrm{lo}} + (c - \theta)] \tag{3}$$

where l_{lo} is the first position in L that is $\geq l$. The dense interval's right endpoint is extended to $l_{\mathrm{dense}} + W_0$, and the left endpoint is updated to $l \leftarrow l_{\mathrm{dense}} + 1$.

Lemma 2 (Correctness of the dense block)**.** *For* l_{dense} *defined by Eq.* (3)*, for any* $l' \in [l,\ l_{\mathrm{dense}}]$*,*

$$|\{t \in L : l' \leq t \leq l' + W_0\}| \geq \theta$$

holds.

The proof is given in Appendix A.

C. Candidate Interval Restriction via Apriori Pruning ($k \geq 2$)

Applying Lemma 1 to each singleton item, any dense-start component of a pattern P is contained within the dense intervals of each item $i \in P$. Using this property, for candidate patterns P we use only *intersections of singleton interval maps*.

a) Candidate interval map.: Let the dense region of singleton item i be

$$\mathcal{R}_\theta(\{i\}) = \{(\boldsymbol{s}_{\mathrm{sp}}, [a, e])\}.$$

Write $\mathcal{R}_\theta(\{i\})[\boldsymbol{s}_{\mathrm{sp}}]$ for the temporal interval sequence belonging to the same spatial start point $\boldsymbol{s}_{\mathrm{sp}}$. The candidate interval map for pattern P is then defined as

$$\mathcal{C}(P, \boldsymbol{s}_{\mathrm{sp}}) = \bigcap_{i \in P} \mathcal{R}_\theta(\{i\})[\boldsymbol{s}_{\mathrm{sp}}]. \tag{4}$$

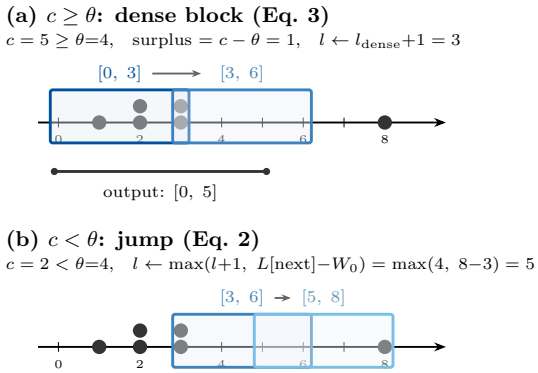


Fig. 1: Duplicate-aware t-axis stride. Given a sorted multiset $L = \{\{1, 2, 2, 3, 3, 8\}\}$ with W_0=3 and θ=4. (a) When $c \geq \theta$, the surplus determines how far the left endpoint can stride in bulk (Eq. (3)). (b) When $c < \theta$, the window jumps to the next position where a new event enters (Eq. (2)). Window color gradient (dark→light blue) indicates the progression of the left endpoint l.

TABLE II: Computational complexity of the proposed method. N_P: number of transactions containing pattern P, $W_{\text{sp}} = \prod_{k=1}^{d-1}(W_k + 1)$, $\widetilde{N}_P = N_P W_{\text{sp}}$, I_P: total number of intervals in the candidate interval map.

Step	Complexity
Construction of L_P	$O(N_P \cdot W_{\text{sp}})$
Sorting of L_P	$O(\widetilde{N}_P \log \widetilde{N}_P)$
Temporal interval computation	$O\left(\sum_{\boldsymbol{s}_{\text{sp}}} \lvert L_P(\boldsymbol{s}_{\text{sp}})\rvert \log \lvert L_P(\boldsymbol{s}_{\text{sp}})\rvert\right)$
Candidate interval map intersection	$O(I_P)$

Here the intersection is the standard interval intersection on temporal intervals. If $\mathcal{C}(P, \boldsymbol{s}_{\text{sp}}) = \emptyset$ for all spatial start points, P cannot have any dense interval and is pruned.

b) Exact re-evaluation.: Only when $\mathcal{C}(P, \boldsymbol{s}_{\text{sp}}) \neq \emptyset$, we scan the transaction sequence once to construct $L_P(\boldsymbol{s}_{\text{sp}})$ and run the candidate-interval version of Algorithm 1 over $\mathcal{C}(P, \boldsymbol{s}_{\text{sp}})$.

D. Overall Algorithm and Complexity

Therefore, the overall complexity faithful to the current implementation is

$$O\left(\sum_{P \in \mathcal{F}} \left(N_P W_{\text{sp}} + \widetilde{N}_P \log \widetilde{N}_P + I_P\right)\right)$$

where $\mathcal{F}$ is the set of candidate patterns actually evaluated. When applied to real data, discretization of continuous coordinates is required as a preprocessing step (§V).

V. Experiments

We verify the scalability and effectiveness of the proposed method. All experiments were implemented in Rust (release build) and executed on Apple M3 (8 cores, 24 GB RAM).

Algorithm 1 Duplicate-aware dense interval computation for a fixed spatial start point

Require: Sorted multiset L, temporal window width W_0, threshold θ
Ensure: Dense interval sequence $\mathcal{R}$

$\mathcal{R} \leftarrow \emptyset$
$l \leftarrow \max(0,\ L[\theta - 1] - W_0)$
open $\leftarrow$ false
while $l \leq L[\text{last}]$ **do**
  $l_{\text{lo}} \leftarrow \text{lower_bound}(L, l)$
  $l_{\text{hi}} \leftarrow \text{upper_bound}(L, l + W_0)$
  $c \leftarrow l_{\text{hi}} - l_{\text{lo}}$
  if $c < \theta$ **then**
    if *open* **then**
      Append current interval to $\mathcal{R}$
      open $\leftarrow$ false
    end if
    if $l_{\text{hi}} = |L|$ **then break**
    end if
    $l \leftarrow \max(l + 1,\ L[l_{\text{hi}}] - W_0)$ ▷ Eq. (2)
  else
    $l_{\text{dense}} \leftarrow L[l_{\text{lo}} + (c - \theta)]$ ▷ Eq. (3)
    $r_{\text{block}} \leftarrow l_{\text{dense}} + W_0$
    Start or extend current interval with (l, r_{block})
    open $\leftarrow$ true
    $l \leftarrow l_{\text{dense}} + 1$
  end if
end while
if *open* **then**
  Append current interval to $\mathcal{R}$
end if
return $\mathcal{R}$

A. EX-1: Method Comparison on Diverse Dense Region Shapes (Synthetic Data)

a) Objective.: Real-world dense regions may take various shapes. This experiment uses synthetic data with four embedding shapes (rectangular, drift, intermittent, and expanding) to verify how well existing methods recover the exact solution defined by the proposed method.

b) Synthetic data.: A T=100 × S=100 grid with 20 embedded GT patterns is used. Four embedding shapes are evaluated: rectangular (Shape A), drift (Shape B), intermittent (Shape C), and expanding (Shape D), with 3 seeds per shape ($4 \times 3 = 12$ datasets in total). Generation details are given in Appendix B.

c) Baseline methods.: Each baseline addresses a fundamentally different formulation, so all tunable parameters were grid-searched to maximize each baseline's score for a fair comparison. The following five methods are used. Colocation / STCP-Miner are used in EX-1-1 (pattern detection); LISA / MZoom / ST-DBSCAN are used in EX-1-2 (region recovery).

- **Colocation** [11]: Spatial colocation mining. PI threshold τ grid-searched.

Algorithm 2 Multi-Dimensional Apriori-Window (interval-based implementation)

Require: d-dimensional transaction sequence $\mathcal{D}$, $(\boldsymbol{W}, \theta)$
Ensure: Dense region $\mathcal{R}_\theta(P)$ for each pattern P

for all $i \in \mathcal{I}$ **do**
  $L_i(\boldsymbol{s}_{\mathrm{sp}}) \leftarrow$ Construct temporal multiset per spatial start point
  $\mathcal{R}_\theta(\{i\}) \leftarrow$ Run Algorithm 1 for each $\boldsymbol{s}_{\mathrm{sp}}$
end for
$\mathcal{F}_1 \leftarrow \{\{i\} : \mathcal{R}_\theta(\{i\}) \neq \emptyset\}$
$k \leftarrow 2$
while $\mathcal{F}_{k-1} \neq \emptyset$ **do**
  $\texttt{cands} \leftarrow$ GENERATECANDIDATES($\mathcal{F}_{k-1}$)
  $\texttt{cands} \leftarrow$ PRUNECANDIDATES(cands, $\mathcal{F}_{k-1}$)
  $\mathcal{F}_k \leftarrow \emptyset$
  for all $P \in \texttt{cands}$ **do**
    $\mathcal{C}(P, \boldsymbol{s}_{\mathrm{sp}}) \leftarrow \bigcap_{i \in P} \mathcal{R}_\theta(\{i\})[\boldsymbol{s}_{\mathrm{sp}}]$ ▷ Eq. (4)
    if $\mathcal{C}(P, \boldsymbol{s}_{\mathrm{sp}}) = \emptyset$ for all $\boldsymbol{s}_{\mathrm{sp}}$ **then**
      continue
    end if
    $L_P(\boldsymbol{s}_{\mathrm{sp}}) \leftarrow$ Construct temporal multiset for pattern P
    $\mathcal{R}_\theta(P) \leftarrow$ Run candidate-interval duplicate-aware solver for each $\boldsymbol{s}_{\mathrm{sp}}$
    if $\mathcal{R}_\theta(P) \neq \emptyset$ **then**
      $\mathcal{F}_k \leftarrow \mathcal{F}_k \cup \{P\}$
    end if
  end for
  $k \leftarrow k + 1$
end while
return $\{\mathcal{R}_\theta(P)\}$

- **STCP-Miner** [12]: Spatio-temporal window extension of Colocation. τ and window width grid-searched.
- **MZoom** [4]: Greedy dense block detection on the occurrence tensor of GT pattern P. Top k grid-searched.
- **LISA** [5]: Occurrence counts of GT pattern P aggregated onto a spatial grid; High-High cells identified by Local Moran's I.
- **ST-DBSCAN** [8]: ε_1, ε_2, min_pts grid-searched on occurrence cells of GT pattern P.

d) Evaluation metrics.: The dense pattern set P^* and dense regions $R^*(P)$ obtained under the sliding-window density definition (Definitions 3–5) for the explicitly embedded patterns serve as the ground truth (GT). Since the proposed method outputs the exact solution of this definition, it achieves F1 $= 1.00$ and Jaccard $= 1.00$ by construction. For EX-1-1, F1 $= 2|\hat{P} \cap P^*|/(|\hat{P}| + |P^*|)$ measures how well each baseline identifies the GT pattern set P^*. For EX-1-2, mean Jaccard $= \frac{1}{|P^*|}\sum_{P \in P^*} |R^*(P) \cap \hat{R}(P)|/|R^*(P) \cup \hat{R}(P)|$ measures how closely each baseline recovers the GT dense region $R^*(P)$.

Note that each baseline solves a fundamentally different problem: colocation methods detect participation-index co-occurrence without defining dense regions; dense subtensor methods output Cartesian-product blocks, not sliding-window regions; clustering methods label point neighborhoods rather than windowed occurrence counts. EX-1 is therefore not a head-to-head competition but a characterization of how closely each method approximates the exact sliding-window solution.

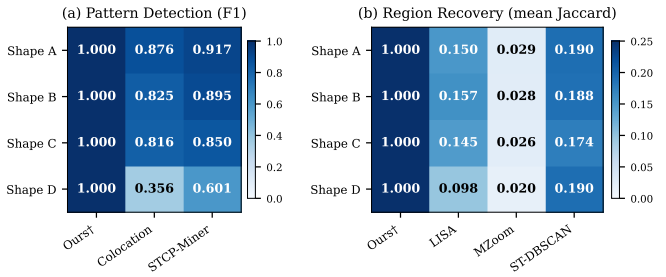


Fig. 2: EX-1 results (3-seed average, p_{embed}=0.25). (a) EX-1-1 F1; (b) EX-1-2 mean Jaccard. GT is the exact solution under the sliding-window definition (Definitions 3–5). †The proposed method outputs the exact solution by construction, hence F1 $= 1.00$ and Jaccard $= 1.00$. Baseline parameters selected by grid search.

e) Proposed method parameters.: Common across all shapes: $W = (20, 5)$, $\theta = 30$.

f) Results.: Fig. 2 shows scores averaged over 3 seeds per shape.

g) Analysis.: EX-1-1 (Pattern detection F1). Colocation performs worst on Shape D (expanding): during the expansion phase, the global PI computation is diluted over the entire observation period, suppressing the participation index. STCP-Miner partially avoids this dilution via temporal windowing, but the approximation gap remains.

EX-1-2 (Region recovery, mean Jaccard). All baselines remain at low scores across all shapes.

LISA collapses occurrences over all time steps, irreversibly discarding temporal structure.

MZoom achieves the lowest scores. Against an S=100-cell grid where each dense band spans only 5 cells, any Cartesian-product block wide enough to cover the full band incurs a severe density drop from the Bernoulli background. Greedy search therefore converges to single-column (width 1) blocks, missing most of the true dense region as false negatives.

ST-DBSCAN labels only occurrence cells as cluster members, systematically missing non-occurrence cells inside dense windows, which constitute the majority of the dense region.

In short, these structural mismatches—not parameter tuning—explain why no baseline approximates the sliding-window definition well across all four shapes.

B. EX-2: Scalability (Synthetic Data)

a) Objective.: We verify how the computation time of the proposed method scales with increasing number of transactions N, and whether the ratio of the number of temporal grid points T to the number of spatial grid points S affects performance at the same N (for $d = 2$, $N = T \times S$).

TABLE III: EX-2: 2D scalability. All cells with Bernoulli background only (no embedded patterns), $N = T \times S$, $\theta = 15$, $W = (24, 4)$. Execution time (seconds). $N = 10^9$ did not complete within 1,000 s.

$T \backslash S$	10	100	1,000	10,000
100	0.009	0.059	0.655	7.18
1,000	0.042	0.470	5.62	64.9
10,000	0.403	4.79	57.5	643
100,000	4.16	53.1	587	>1000

b) Synthetic data generation.: On a $d = 2$ lattice $\{0, \ldots, T-1\} \times \{0, \ldots, S-1\}$, each item is independently sampled from Bernoulli(p_{item}=0.10) for all cells to form one basket. Empty cells are assigned item 1 by default, ensuring exactly one transaction per cell, so that $N = T \times S$ (generation details in Appendix C).

c) Proposed method parameters.: $|\mathcal{I}| = 20$, $W = (24, 4)$, and $\theta = 15$ were fixed.

d) Rationale for experimental axes.: Since linear scaling with $|\mathcal{I}|$ and p_{item} was confirmed in the 1D version [3], and the effect of d is absorbed into $N = T \times S^{d-1}$, we fix $d = 2$ and vary T and S.

e) Experimental design.: $S \in \{10,\ 100,\ 1{,}000,\ 10{,}000\}$ and $T \in \{100,\ 1{,}000,\ 10{,}000,\ 100{,}000\}$ yield $4 \times 4 = 16$ configurations. $N = T \times S$ ranges from 10^3 to 10^9; same-N configurations with differing T/S ratios isolate each axis's effect.

f) Results.: Table III shows the execution times.

g) Analysis.: At $N = 10^3$, 0.009 seconds; at $N = 10^8$, 587–643 seconds: for a 10^5-fold increase in N, execution time increases only by a factor of approximately 6.5×10^4. $N = 10^9$ did not complete within 1,000 seconds (timeout).

Even at the same N, the T/S ratio causes differences: at $N = 10^7$, T=100,000, S=100 takes 53.1 s while T=1,000, S=10,000 takes 64.9 s (1.22×). Larger T enables more effective gap-skipping via the t-axis stride (Lemma 2), while larger S increases the number of spatial start points.

Overall, execution time scales approximately linearly with N, and 10^8 transactions can be processed within 10 minutes.

C. EX-3: Application to Real-World Data (d=3–4)

a) Objective.: We apply the proposed method to real-world data from diverse domains and confirm that the detected patterns and dense regions yield interpretable insights consistent with domain knowledge.

b) Datasets and results.: Two crime report datasets (Chicago Crime [22], NYPD Crime [23]), citizen complaints (NYC 311 [24]), and an earthquake catalog (USGS Earthquake [25]) are used, spanning three domains and d=3–4 dimensions. Time is discretized into weekly bins and spatial coordinates into equal-width bins, converting each cell's events into a transaction. Dataset details and quantitative results are given in Appendix D.

c) Findings.: In Chicago Crime, ASSAULT + BATTERY forms chronic dense windows in south and west side districts, and three-item patterns (e.g., THEFT + CRIMINAL DAMAGE + BATTERY) pinpoint compound hotspots missed by single-category analysis. In NYPD Crime, compound co-occurrences of felonies and misdemeanors are captured at the borough and weekly granularity. In NYC 311, combinations of noise complaints and illegal parking repeatedly appear in residential neighborhoods. In USGS Earthquake, 4-dimensional analysis including the depth axis distinguishes dense patterns of shallow crustal earthquakes from intermediate-depth intraslab events. In all cases, the proposed method outputs *which* itemsets are dense and *when and where* as concrete spatio-temporal windows, providing interpretable results for domain experts. Detailed case studies are given in Appendix D.

VI. Conclusion

This paper proposed **Multi-Dimensional Apriori-Window**, which extends Apriori-Window [3] to arbitrary d-dimensional grids. Through three techniques—one-dimensional multiset reduction per fixed spatial start point, duplicate-aware t-axis stride, and singleton interval map intersection pruning—the method exactly enumerates all dense patterns together with their arbitrarily-shaped dense regions.

Experiments demonstrated that: (1) on synthetic data with four embedding shapes, the proposed method outputs the exact solution, while existing methods achieved at most F1 = 0.917 and Jaccard = 0.190; (2) linear scaling was confirmed for $N = 10^3$–10^8, processing 10^8 transactions within 10 minutes; (3) on d=3–4 real-world data from crime, complaints, and earthquake domains, the method yielded interpretable insights consistent with domain knowledge, such as compound crime hotspots and depth-stratified seismic clusters.

a) Limitations.: (1) The method assumes grid discretization, and the choice of bin width and $\boldsymbol{W}$ affects results. (2) As W_{sp} or d increases, the $O(N \cdot W_{\text{sp}})$ cost of multiset construction grows, requiring dimensionality reduction or coarse binning in high-dimensional settings. (3) The method is specialized for detection and description; causal explanation and association with external events are outside its scope.

b) Future work.: (1) *Adaptive parameter selection*: A framework that automatically adjusts $\boldsymbol{W}$ and θ based on local data statistics (density distributions, spatial autocorrelation) is practically important. (2) *Further acceleration*: Construction of temporal multisets per spatial start point is the main computational bottleneck. Prefix-sum-based $O(1)$ window aggregation and incremental update along spatial axes (slice differencing) to eliminate redundant computation are promising.

Acknowledgment

The authors acknowledge that portions of the text in this paper were generated with Anthropic's large language model Claude [26].

REFERENCES

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proceedings of VLDB*, 1994, pp. 487–499.

[2] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 1–12, 2000.

[3] Anonymous, "Exact mining of dense patterns via direct evaluation of interval-local frequency with sliding windows," in *Proceedings of DSAA*, 2025, under review.

[4] K. Shin, B. Hooi, and C. Faloutsos, "M-Zoom: Fast dense-block detection in tensors with quality guarantees," in *Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2016, pp. 264–280.

[5] L. Anselin, "Local indicators of spatial association—LISA," *Geographical Analysis*, vol. 27, no. 2, pp. 93–115, 1995.

[6] J. K. Ord and A. Getis, "Local spatial autocorrelation statistics: distributional issues and an application," *Geographical Analysis*, vol. 27, no. 4, pp. 286–306, 1995.

[7] M. Kulldorff, "A spatial scan statistic," *Communications in Statistics-Theory and Methods*, vol. 26, no. 6, pp. 1481–1496, 1997.

[8] D. Birant and A. Kut, "ST-DBSCAN: An algorithm for clustering spatial–temporal data," *Data & Knowledge Engineering*, vol. 60, no. 1, pp. 208–221, 2007.

[9] A. K. Mahanta, F. A. Mazarbhuiya, and H. K. Baruah, "Finding locally and periodically frequent sets and periodic association rules," in *Proc. PReMI*, 2005, pp. 576–582.

[10] P. Fournier-Viger, P. Yang, R. U. Kiran, S. Ventura, and J. M. Luna, "Mining local periodic patterns in a discrete sequence," *Information Sciences*, vol. 544, pp. 519–548, 2021.

[11] S. Shekhar and Y. Huang, "Discovering spatial co-location patterns: A summary of results," in *Advances in Spatial and Temporal Databases (SSTD)*, ser. LNCS, vol. 2121. Springer, 2001, pp. 236–256.

[12] F. Qian, L. Yin, Q. He, and J. He, "Mining spatio-temporal co-location patterns with weighted sliding window," in *2009 IEEE International Conference on Intelligent Computing and Intelligent Systems (ICI-CISYS)*, vol. 3. IEEE, 2009, pp. 181–185.

[13] M. Jiang, A. Beutel, P. Cui, B. Hooi, S. Yang, and C. Faloutsos, "CrossSpot: A general suspiciousness metric for dense blocks in multimodal data," in *Proceedings of the 2015 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2015, pp. 781–786.

[14] K. Shin, B. Hooi, and C. Faloutsos, "Fast, accurate and flexible algorithms for dense subtensor mining," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 12, no. 3, pp. 28:1–28:30, 2018.

[15] K. Shin, B. Hooi, J. Kim, and C. Faloutsos, "D-cube: Dense-block detection in terabyte-scale tensors," in *Proceedings of WSDM*, 2017, pp. 681–689.

[16] ——, "Densealert: Incremental dense-subtensor detection in tensor streams," in *Proceedings of KDD*, 2017, pp. 1057–1066.

[17] W. Feng, S. Liu, and X. Cheng, "CatchCore: Catching hierarchical dense subtensor," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2020, pp. 156–172.

[18] W. Feng, S. Liu, D. Koutra, H. Shen, and X. Cheng, "SpecGreedy: Unified dense subgraph detection," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2020, pp. 181–197.

[19] M. Kulldorff, "Prospective time periodic geographical disease surveillance using a scan statistic," *Journal of the Royal Statistical Society: Series A*, vol. 164, no. 1, pp. 61–72, 2001.

[20] T. Tango and K. Takahashi, "A flexibly shaped spatial scan statistic for detecting clusters," *International Journal of Health Geographics*, vol. 4, no. 1, p. 11, 2005.

[21] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of KDD*, 1996, pp. 226–231.

[22] City of Chicago, "Crimes – 2001 to present," Chicago Data Portal. https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2, 2024, accessed 2026.

[23] New York City Police Department, "NYPD complaint data historic," NYC Open Data. https://data.cityofnewyork.us/Public-Safety/NYPD-Complaint-Data-Historic/qgea-i56i, 2024, accessed 2026.

[24] NYC Office of Technology and Innovation, "311 service requests from 2020 to present," NYC Open Data. https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9, 2024, accessed 2026.

[25] U.S. Geological Survey, "USGS earthquake catalog (ComCat)," https://earthquake.usgs.gov/earthquakes/search/, 2021, m2.5+, 2000–2020, accessed 2026.

[26] Anthropic, "Claude," https://claude.ai, 2025, accessed: 2026.

APPENDIX A
PROOFS OF LEMMAS

Proof of Lemma 1. For any transaction n, if $P \subseteq d_n$ then $Q \subseteq d_n$. Therefore, for any fixed spatial start vector $\boldsymbol{s}_{\mathrm{sp}}$, each element of $L_P(\boldsymbol{s}_{\mathrm{sp}})$ is also contained in $L_Q(\boldsymbol{s}_{\mathrm{sp}})$. Hence, for any s_0, $S_P(s_0, \boldsymbol{s}_{\mathrm{sp}}) \leq S_Q(s_0, \boldsymbol{s}_{\mathrm{sp}})$ holds. □

Proof of Lemma 2. Consider the rightmost θ events within the current window $[l, l + W_0]$. The leftmost of these is $L[l_{\mathrm{lo}} + (c - \theta)] = l_{\mathrm{dense}}$. For any $l' \leq l_{\mathrm{dense}}$, all θ events are $\geq l'$, and since $l' \geq l$, we have $l' + W_0 \geq l + W_0$. Therefore, these events do not leave the current window $[l, l + W_0]$ and are all contained in $[l', l' + W_0]$ as well. Hence, at least θ events remain within the window at l'. □

APPENDIX B
EX-1 SYNTHETIC DATA GENERATION DETAILS

a) Background data generation.: One transaction is placed at every grid point of a T=100 × S=100 grid. With $|\mathcal{I}|$=40 items, each item is independently sampled from Bernoulli(p_{item}=0.15) per cell to form the background. Cells where all items are absent are assigned item 1 by default.

b) Pattern embedding.: GT patterns $P_k = \{2k-1,\ 2k\}$ ($k = 1, \ldots, 20$) are assigned to spatial bands $s \in [5(k-1),\ 5(k-1)+4]$ (width 5 cells). Within cells belonging to the embedding region of each shape, item pair P_k is co-embedded independently with Bernoulli(p_{embed}=0.25).

c) Embedding shapes.: Each shape specifies the spatio-temporal distribution of pattern P_k within its spatial band ($b_s = 5(k-1)$, $b_e = b_s + 4$).

- **Shape A (rectangular)**: Uniformly embedded in all cells with $t \in [30, 70]$ and $s \in [b_s, b_e]$. The simplest fixed rectangular region in both time and space.
- **Shape B (drift)**: At each time step $t \in [30, 70]$, a band of width 5 is centered at $s_c = \lfloor b_s + (t-30)/40 \cdot 2 \rceil$ (linearly shifting from b_s to b_s+2). The dense band drifts smoothly along the spatial axis over time.
- **Shape C (intermittent)**: $t \in [15, 35] \cup [65, 85]$, $s \in [b_s, b_e]$. Dense activity occurs in the same spatial band during two independent time intervals.
- **Shape D (expanding)**: $t \in [30, 40]$: center cell b_s+2 only (width 1); $t \in [41, 55]$: $s \in [b_s+1, b_e-1]$ (width 3); $t \in [56, 70]$: $s \in [b_s, b_e]$ (width 5). The spatial extent grows stepwise as time progresses.

d) Evaluation setup.: Three random seeds (seed ∈ $\{42, 55, 68\}$) are used per shape, giving $4 \times 3 = 12$ datasets in total. The proposed method uses $W = (20, 5)$ and $\theta = 30$ uniformly across all shapes. Grid search parameter ranges are shown in Table IV.

APPENDIX C
EX-2 SYNTHETIC DATA GENERATION DETAILS

In EX-2, synthetic data is generated by placing one transaction at **every** grid point of a $d = 2$ grid $\{0, \ldots, T-1\} \times \{0, \ldots, S-1\}$. For each transaction, each item is independently sampled from Bernoulli(p_{item}) with vocabulary size $|\mathcal{I}| = 20$ and background occurrence probability $p_{\mathrm{item}} = 0.10$. Cells where all items are absent are assigned item 1 by default, guaranteeing exactly one transaction per cell. This ensures the number of transactions is exactly $N = T \times S$.

TABLE IV: Grid search parameter ranges for EX-1 baseline methods.

Method	Grid search targets
Colocation	PI threshold $\tau \in \{0.05, 0.07, \ldots, 0.49\}$ (23 values)
STCP-Miner	PI threshold (same) × window width $W_t \in \{4, 13, 26, 54\}$
MZoom	Top block count $k \in \{1, 2, 3, 5\}$
ST-DBSCAN	$\varepsilon_1 \in \{1, 2, 3\}$, $\varepsilon_2 \in \{3, 7, 15\}$, min_pts $\in \{2, 5\}$

No embedded patterns are injected. Dense patterns arise naturally from statistical fluctuations of the Bernoulli background. Preliminary calibration confirmed that a sufficient number of patterns are detected at threshold $\theta = 15$ across all configurations.

As discussed in §V-B, a separate dimensionality scalability experiment is unnecessary because the growth of $N = T \times S^{d-1}$ with increasing d in a d-dimensional grid is absorbed into N-scalability.

APPENDIX D
REAL DATASET DETAILS

a) Dataset overview.: Details of the four real-world datasets used in EX-3 (§V-C) are given below.

- **Chicago Crime** [22]: Crime report data published by the City of Chicago. Includes date/time, latitude, longitude, and crime type (33 types). Time discretized into weekly bins; latitude and longitude into 0.02° equal-width bins (grid size $261 \times 20 \times 21$).
- **NYPD Crime** [23]: Crime report data published by the New York City Police Department. Includes crime type (69 types). Weekly bins, 0.02° spatial bins (grid size $261 \times 43 \times 57$).
- **NYC 311** [24]: Citizen complaint (311) data from New York City. Includes complaint type (232 types). Weekly bins, 0.02° spatial bins (grid size $261 \times 43 \times 57$).
- **USGS Earthquake** [25]: Earthquake catalog published by the United States Geological Survey. Magnitude discretized into 5 bands and used as items. This is the only 4-dimensional dataset (time × latitude × longitude × depth), with grid size $1{,}096 \times 20 \times 20 \times 4$.

b) Experimental parameters.: Maximum pattern length $|P| \leq 3$ was used for all datasets. The three 3-dimensional datasets (Chicago/NYPD/NYC 311) share temporal window $W_0 = 4$ weeks and spatial windows $W_k = 3$ bins ($k = 1, 2$), with only threshold θ adjusted per dataset. USGS Earthquake used $W_0 = 13$ weeks, $W_k = 2$ bins ($k = 1, 2, 3$), and $\theta = 2$.

c) Analysis results.: Table VI shows the number of detected patterns and execution time for each dataset. Pattern counts increase with item vocabulary size, and execution times

TABLE V: Experimental datasets. N: number of transactions (non-empty cells), #Items: number of item types, avgℓ: average items per transaction, d: dimensionality.

Dataset	N	#Items	avgℓ	d
Chicago Crime [22]	47,217	33	7.59	3
NYPD Crime [23]	189,481	69	5.67	3
NYC 311 [24]	135,490	232	18.14	3
USGS Earthquake [25]	13,151	5	1.17	4

range from 0.5 to 108.7 seconds in proportion to data scale and vocabulary. Detailed findings for each dataset are given below.

TABLE VI: Detection results of the proposed method on real datasets. θ: density threshold, Total: number of patterns (all sizes), time: execution time (seconds).

Dataset	W	θ	Total	time (s)
Chicago Crime	$(4, 3, 3)$	3	2,835	9.7
NYPD Crime	$(4, 3, 3)$	5	4,478	38.2
NYC 311	$(4, 3, 3)$	35	5,130	108.7
USGS Earthq.	$(13, 2, 2, 2)$	2	18	0.5

Chicago Crime. Analysis of five years (261 weeks) of Chicago-wide crime data on a 3-dimensional grid yields 2,835 patterns across 33 crime categories. Pairs such as ASSAULT + BATTERY appear as dense windows in multiple districts on the south and west sides, describing chronic crime concentration over $4\,\mathrm{wks} \times 3\,\mathrm{bins} \times 3\,\mathrm{bins}$ windows. THEFT + CRIMINAL DAMAGE co-concentrates in commercial and urban areas as weekly dense windows. Three-item patterns such as THEFT + CRIMINAL DAMAGE + BATTERY precisely pinpoint locations and periods where multiple crime types co-concentrate within the same spatio-temporal window, revealing compound hotspots that simple per-category frequency analysis would miss.

NYPD Crime. Five years of New York City-wide data yield 4,478 patterns across 69 crime categories. The richer vocabulary (vs. Chicago's 33) produces more patterns, capturing compound co-occurrences of felonies (e.g., Felony Assault, Robbery) and misdemeanors (e.g., Petit Larceny, Criminal Mischief) at the borough and weekly granularity. Many individual crime types are individually low-frequency, yet locally concentrate as combinations within specific neighborhoods and periods; the proposed method explicitly describes this compound crime structure that simple frequency rankings cannot surface.

NYC 311. With 232 complaint categories—the largest vocabulary—5,130 patterns (the most) are detected. Combinations of Noise – Residential and Noise – Street/Sidewalk with Illegal Parking repeatedly form dense windows in residential neighborhoods, reflecting compound quality-of-life degradation. Summer months exhibit an increased tendency for three-complaint patterns (construction noise + sanitation + illegal parking) to concentrate in specific boroughs, capturing seasonal compound problems. The large vocabulary makes co-occurrence patterns difficult to find by per-complaint frequency analysis alone; spatial–temporal localization makes these results actionable for borough- and week-level policy planning.

USGS Earthquake. Twenty-one years (1,096 weeks) of earthquake catalog data are analyzed on a 4-dimensional grid (time $\times$ latitude $\times$ longitude $\times$ depth), yielding 18 patterns across 5 magnitude bands. Co-occurrence of M3.0–4.0 and M4.0–5.0 within a specific latitude–longitude–depth–time window indicates concentration of multiple magnitude bands consistent with aftershock sequences or earthquake swarms following larger events. Including the depth axis allows the method to distinguish shallow crustal earthquakes ($\lesssim$30 km) from intermediate-depth intraslab events, suggesting correspondence with fault type and subduction-zone structure. With only 5 items, computational cost is minimal, demonstrating that the proposed method operates in near-real time even in a 4-dimensional high-dimensional setting.